

Sentiment Analysis of Mobile Banking App Reviews

A Reproducible Study: R to Python, Old Data to New Data, Lexicon to Transformer

Chi Dao I Putu Agastya Harta Pratama Si Tang Lin
Zuzanna Uljasz

2026-06-01

Table of contents

0.1	Summary	2
0.2	1. Introduction and Motivation	2
0.2.1	1.1 Research Context	2
0.2.2	1.2 Inspiration and Source Study	2
0.3	2. Data	3
0.3.1	2.1 Exploratory Data Analysis of the Original Dataset (Old Data)	3
0.3.2	2.2 Exploratory Data Analysis of the New Dataset (New Data)	5
0.3.3	2.3 Data Preprocessing	5
0.4	3. Methodology	5
0.4.1	3.1 Lexicon-Based Sentiment Analysis	5
0.4.2	3.2 Bigram Analysis and Negation Correction	6
0.4.3	3.3 Transformer-Based Sentiment Analysis	6
0.5	4.1 Results Python vs R	7
0.6	4.2 Results Old Data vs New Data - Lexicon Analysis	7
0.7	4.2 Results Old Data vs New Data - Transformer Analysis	7
0.8	4.2 Results AFINN vs BERT - New Data	7
1	5 Discussion	11
1.1	Reproducibility Assessment	11
2	6 Limitations	11
3	7 Generative AI Disclosure	11

0.1 Summary

This report documents a reproducible research project that translates an original R-based sentiment analysis of mobile banking app reviews into Python, extends it with newly scraped data, and augments the lexicon-based methods with a transformer (BERT) approach. The project covers five major US banks: Bank of America, Chase, Citi, Marcus by Goldman Sachs, and Wells Fargo across the Apple App Store (iOS) and Google Play (Android). The core research questions are: (1) Can the original R findings be faithfully reproduced in Python? (2) Do the results change with a fresh data scrape? (3) Does a transformer model produce sentiment classifications consistent with the lexicon approach?

0.2 1. Introduction and Motivation

0.2.1 1.1 Research Context

Customer reviews on mobile app stores represent a rich, continuously updated source of unstructured opinion data. For the banking sector, app reviews offer direct evidence of customer satisfaction, pain points, and competitive positioning — feedback that often offers a faster point of view and is more granular than traditional survey methods. Sentiment analysis of these reviews can help researchers and practitioners understand how user experience varies across institutions and platforms, and whether negative signals co-occur with specific feature complaints or rating patterns.

There are two motivations for this study. Academically, it provides an opportunity to practice reproducible research: taking an existing empirical study, replicating it in a different computational environment, and evaluating whether the results remain the same. Practically, it investigates whether mobile banking sentiment has shifted between the original data collection window and a fresh scrape in 2026, and whether more sophisticated deep learning methods agree with classic dictionary-based approaches.

0.2.2 1.2 Inspiration and Source Study

The project is inspired by the tidytext-based workflow popularised by @silge2017text, which applies lexicon-based sentiment scoring to tokenised text in a tidy data format. The original analysis follows this approach: it imports and cleans review data, performs unigram sentiment analysis using three lexicons (Bing, AFINN, NRC), constructs bigrams with negator-aware scoring, and evaluates results statistically. The goal of this project is to demonstrate that

these findings are reproducible — the same pipeline, implemented in Python with equivalent libraries, produces consistent conclusions — and then to extend the study in two directions: new data and a transformer model.

0.3 2. Data

Dataset is obtained with the means of scrapping reviews from the App Store and Google Play for the five major banks in US. The original dataset was collected in 2023, while the new dataset was scraped in mid-2026. Both datasets contain reviews in English, with metadata including star ratings, review text, date, platform, and storefront. The data is stored as CSV files per bank and platform, which are combined and preprocessed for analysis.

The major banks included in the dataset are some of the largest US financial institutions, which are: Bank of America, Chase, Citi, Marcus by Goldman Sachs, and Wells Fargo. Each has a significant mobile banking user base. The platforms covered are the two dominant mobile ecosystems: Apple’s App Store (iOS) and Google’s Play Store (Android). This allows for cross-platform comparisons and insights into whether user sentiment differs by operating system.

The method of scrapping differs for each platform due to their distinct APIs and structures. For the iOS App Store, the script uses Python’s requests library to directly parse Apple’s publicly accessible XML/JSON RSS feed endpoints, which are ordered by the most recent submissions. For the Google Play Store, it leverages an open-source web scraping library (google-play-scraper) that mimics browser requests to dynamically fetch and page through raw review payloads using continuation tokens. Both approaches process these external data streams in real time, converting the raw JSON responses into structured Pandas DataFrames before exporting them to CSV files.

0.3.1 2.1 Exploratory Data Analysis of the Original Dataset (Old Data)

The original dataset was collected in 2023 using a combination of web scraping and API access methods. It includes reviews from the same five banks and both platforms, but covers a different time period than the new data. The dataset is stored as CSV files, one per bank and platform, with columns for review text, star rating, date, and other metadata. The original analysis applied lexicon-based sentiment scoring to this dataset to derive insights about user sentiment and its relationship to star ratings.

Table 1: Overview of the original dataset

```
from pathlib import Path
import pandas as pd

DATA_OLD = Path("Dataset/data_old_approach")
raw_old = pd.concat(
    [pd.read_csv(p, dtype=str).assign(source_file=p.name)
      for p in DATA_OLD.glob("*.csv")],
    ignore_index=True
)

pd.DataFrame({
    "Total reviews": [len(raw_old)],
    "Banks":          [raw_old["source_file"].nunique()],
    "Platforms":      [raw_old["platform"].nunique()],
    "Date range": [
        f"{pd.to_datetime(raw_old['date'], errors='coerce').min().date()} - "
        f"{pd.to_datetime(raw_old['date'], errors='coerce').max().date()}"
    ]
})
```

0.3.2 2.2 Exploratory Data Analysis of the New Dataset (New Data)

The new dataset was collected using a dedicated scraper (`Dataset/scrapper.ipynb`) targeting the same five banks and both platforms. The scraping window covers a more recent period (2024-2026) than the original data. While the column structure is identical to the old dataset, the new data captures a different temporal slice of user opinions and may reflect changes in app features, service quality, or competitive dynamics that occurred after the original collection window.

0.3.3 2.3 Data Preprocessing

Preprocessing follows the same logical steps in both the R original and the Python replications:

1. **Import and combine** — all per-bank CSVs are concatenated into a single dataframe
2. **Bank and platform standardisation** — filenames are parsed to extract bank names; platform codes (`app_store` → `iOS`, `google_play` → `Android`) are mapped to readable labels
3. **Type coercion** — `rating` is cast to numeric; `date` is parsed to datetime
4. **Language detection** — reviews not detected as English are removed (using `langdetect` in Python)
5. **Raw text construction** — `title` and `review` fields are concatenated into `raw_text`
6. **Tokenisation and stopword removal** — words are extracted using regex; a custom stopword list (NLTK English stopwords + domain terms: *app, bank, banking, mobile, iphone, apple, android, phone*) is applied
7. **Bigram construction** — adjacent word pairs are extracted from cleaned text, with negators (*not, never, no, cannot*, etc.) explicitly excluded from the stopword filter so they are preserved in bigrams

For the transformer approach, an additional cleaning step keeps punctuation intact (since BERT tokenisers handle it natively) and removes URLs.

0.4 3. Methodology

0.4.1 3.1 Lexicon-Based Sentiment Analysis

Three lexicons are applied to the filtered tokens. Scores are computed at the review level and disaggregated by bank and platform.

Bing (Opinion Lexicon): A binary lexicon classifying words as positive or negative [hu2004mining]. Review sentiment is the count of positive words minus negative words.

AFINN: An integer-scored lexicon (−5 to +5) [nielsen2011new]. Review scores are the sum of matched word scores. Pearson correlation between AFINN scores and star ratings serves as a validity check.

NRC Emotion Lexicon: Developed by mohammad2013crowdsourcing, this lexicon maps words to 8 emotion categories (anger, anticipation, disgust, fear, joy, sadness, surprise, trust) and 2 polarity categories. One word may belong to multiple categories, enabling nuanced emotion profiling.

0.4.2 3.2 Bigram Analysis and Negation Correction

Unigram methods cannot handle negation: *not good* is scored as positive because *good* carries a positive AFINN score. To quantify this limitation, bigrams are constructed from the raw review text, with negators explicitly retained in the stopwords filter. A set of 22 negators is defined (e.g., *no*, *not*, *n't*, *never*, *cannot*). Where the first token of a bigram is a negator and the second has an AFINN score, the score is sign-flipped. The magnitude of the resulting correction is reported by star-rating group.

0.4.3 3.3 Transformer-Based Sentiment Analysis

As a complement to the lexicon methods, the pre-trained model `nlptown/bert-base-multilingual-uncased-sentiment` [nlptown2019] is applied via the Hugging Face `transformers` pipeline. The model predicts a 1–5 star rating for each review; predictions are normalised to a continuous sentiment score:

$$\text{bert_score} = \frac{\text{bert_stars} - 1}{4} \in [0, 1]$$

Model validity is assessed by (1) Pearson correlation between `bert_score` and the actual star rating (95% CI via Fisher’s *z*-transformation), (2) boxplots of `bert_score` stratified by star rating, bank, and platform, (3) exact star-level agreement and a full confusion matrix, and (4) a Mann–Whitney U test comparing platform distributions.

Table 2: Pearson correlation: AFINN score vs star rating

```
from scipy import stats

cor_data = df_plot[["rating", "afinn_sum"]].dropna()
r, p = stats.pearsonr(cor_data["rating"], cor_data["afinn_sum"])
print(f"r = {r:.4f}, p = {p:.4e}, n = {len(cor_data):,}")
```

0.5 4.1 Results Python vs R

0.6 4.2 Results Old Data vs New Data - Lexicon Analysis

0.7 4.2 Results Old Data vs New Data - Transformer Analysis

0.8 4.2 Results AFINN vs BERT - New Data

```
import matplotlib.pyplot as plt

raw_old["rating"] = pd.to_numeric(raw_old["rating"], errors="coerce")
rating_counts = (
    raw_old.groupby(["platform", "rating"])
    .size().reset_index(name="n")
)
platforms = sorted(rating_counts["platform"].dropna().unique())
fig, axes = plt.subplots(1, len(platforms), figsize=(10, 4), sharey=True)
for ax, plat in zip(axes, platforms):
    data = rating_counts[rating_counts["platform"] == plat]
    ax.bar(data["rating"], data["n"], color="steelblue")
    ax.set_title(plat)
    ax.set_xlabel("Star rating")
    ax.set_ylabel("Number of reviews")
fig.suptitle("Rating distribution by platform")
plt.tight_layout()
plt.show()
```

Figure 1



```

import nltk, re, numpy as np
from nltk.corpus import opinion_lexicon, stopwords
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

nltk.download("opinion_lexicon", quiet=True)
nltk.download("stopwords", quiet=True)

PROCESSED_OLD = Path("Dataset/data_old_approach/processed_data")
lexicon_df = pd.read_csv(PROCESSED_OLD / "old_preprocessed_lexicon.csv")

custom_stop = set(stopwords.words("english")) | {
    "app", "bank", "banking", "mobile", "iphone", "apple", "android", "phone"
}
pos_words = set(opinion_lexicon.positive())
neg_words = set(opinion_lexicon.negative())

df_english = lexicon_df.copy().reset_index(drop=True)
df_english["review_id"] = df_english.index

reviews_tokens = df_english.copy()
reviews_tokens["word"] = (
    reviews_tokens["raw_text"].fillna("").str.lower()
    .apply(lambda x: re.findall(r"\b[a-z]+\b", x))
)
reviews_tokens = reviews_tokens.explode("word").dropna(subset=["word"])
reviews_after_sw = reviews_tokens[~reviews_tokens["word"].isin(custom_stop)].copy()

bing = pd.DataFrame({
    "word": list(pos_words) + list(neg_words),
    "sentiment": ["positive"] * len(pos_words) + ["negative"] * len(neg_words)
})
bing_words = reviews_after_sw.merge(bing, on="word", how="inner")

counts = bing_words["sentiment"].value_counts().reset_index()
counts.columns = ["sentiment", "n"]

fig, ax = plt.subplots(figsize=(5, 4))
ax.bar(counts["sentiment"], counts["n"], color=["steelblue", "salmon"])
ax.set_title("Distribution of Bing sentiment words")
ax.set_xlabel("Sentiment")
ax.set_ylabel("Word count")
plt.tight_layout()
plt.show()

```

Figure 2


```

import urllib.request

afinn_path = Path("AFINN-111.txt")
if not afinn_path.exists():
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/fnielsen/afinn/master/afinn/data/AFINN-111.txt",
        afinn_path
    )
afinn = pd.read_csv(afinn_path, sep="\t", names=["word", "value"])
afinn["word"] = afinn["word"].str.lower()

afinn_words = reviews_after_sw.merge(afinn, on="word", how="inner")
afinn_review = (
    afinn_words.groupby(["bank", "platform", "review_id"])
    .agg(afinn_sum=("value", "sum"), afinn_mean=("value", "mean"),
        n_afinn=("value", "count")).reset_index()
)
df_afinn = df_english.copy()
df_afinn["rating"] = pd.to_numeric(df_afinn["rating"], errors="coerce")
df_afinn = df_afinn.merge(afinn_review, on=["bank", "platform", "review_id"], how="left")

df_plot = df_afinn.dropna(subset=["rating", "afinn_sum"])
ratings = sorted(df_plot["rating"].unique())
groups = [df_plot[df_plot["rating"] == r]["afinn_sum"] for r in ratings]

fig, ax = plt.subplots(figsize=(7, 4))
bp = ax.boxplot(groups, labels=[int(r) for r in ratings], patch_artist=True)
for patch in bp["boxes"]:
    patch.set_facecolor("lightblue"); patch.set_alpha(0.7)
ax.axhline(0, linestyle="--")
ax.set_title("AFINN sentiment by star rating")
ax.set_xlabel("Star rating")
ax.set_ylabel("AFINN sum score")
plt.tight_layout()
plt.show()

```

Figure 3

```

nrc_path = Path("NRC-Emotion-Lexicon-Wordlevel-v0.92.txt")
if not nrc_path.exists():
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/dinbav/LeXmo/master/NRC-Emotion-Lexicon-Wordlevel-v0.92.txt",
        nrc_path
    )
nrc = pd.read_csv(nrc_path, sep="\t", names=["word", "sentiment", "association"])
nrc = nrc[nrc["association"] == 1][["word", "sentiment"]].copy()
nrc["word"] = nrc["word"].str.lower()

nrc_words = reviews_after_sw.merge(nrc, on="word", how="inner")
nrc_by_bank = nrc_words.groupby(["bank", "sentiment"]).size().reset_index(name="n")
nrc_by_bank["pct"] = nrc_by_bank.groupby("bank")["n"].transform(lambda x: x / x.sum())

banks = sorted(nrc_by_bank["bank"].unique())
fig, axes = plt.subplots(1, len(banks), figsize=(5 * len(banks), 5), sharey=True)
for ax, bank in zip(axes, banks):
    data = nrc_by_bank[nrc_by_bank["bank"] == bank].sort_values("pct")
    ax.barh(data["sentiment"], data["pct"])
    ax.set_title(bank, fontsize=9)
    ax.set_xlabel("Proportion")
fig.suptitle("NRC emotion composition by bank")
plt.tight_layout()
plt.show()

```

Figure 4

Table 3: Statistical evaluation

```

from scipy import stats

ios      = df_afinn[df_afinn["platform"] == "iOS"]["afinn_sum"].dropna()
android  = df_afinn[df_afinn["platform"] == "Android"]["afinn_sum"].dropna()
u_stat, p_mw = stats.mannwhitneyu(ios, android, alternative="two-sided")

print(f"Mann-Whitney U (iOS vs Android): U = {u_stat:.0f}, p = {p_mw:.4e}")
print(f"iOS reviews: {len(ios):,} | Android reviews: {len(android):,}")

```

1 5 Discussion

1.1 Reproducibility Assessment

Something about environment in python Overall assesment about reproducibility, minor differences, but overall conclusions hold Slight conclusions about the original study’s robustness to language and implementation choices

The R-to-Python translation demonstrates that the core analytical conclusions of the original study are reproducible across languages. Minor numerical differences arise from:

- **Language detection:** `langdetect` vs `cld3` use different underlying models; the set of English-classified reviews may differ slightly
- **Stopword lists:** `tidytext::stop_words` differs from NLTK’s English list
- **Bing lexicon size:** NLTK `opinion_lexicon` covers more words than the Bing lexicon embedded in `tidytext`

These are expected cross-language reproducibility gaps and do not undermine the directional conclusions.

2 6 Limitations

- **Lexicon coverage:** All three lexicons were built for general English, not financial app reviews — domain terms (*overdraft*, *Zelle*, *wire transfer*) may be absent
- **Negation scope:** The bigram correction handles only two-word negation; longer-range negation is not captured
- **Temporal confounding:** Old and new data cover different time windows, so observed differences may reflect temporal trends, not genuine quality changes
- **Label imbalance:** App Store reviews skew heavily toward 5 stars; without class-weighting, the transformer may over-predict “positive”

3 7 Generative AI Disclosure

In accordance with course policy (dr Jakub Michańków, Reproducible Research 2026), generative AI (Claude AI) was used to assist with coding tasks, refine the code.

All analytical decisions, interpretations, and research design are the team’s own work.

4 References